

PRAKTIKUM SISTEM OPERASI

MODUL XI
(Memori Xinu)



Oleh:

Farhan Kurniawan (2311104073)

PROGRAM STUDI REKAYASA PERANGKAT LUNAK
DIREKTORAT KAMPUS PURWOKERTO
UNIVERSITAS TELKOM
2026

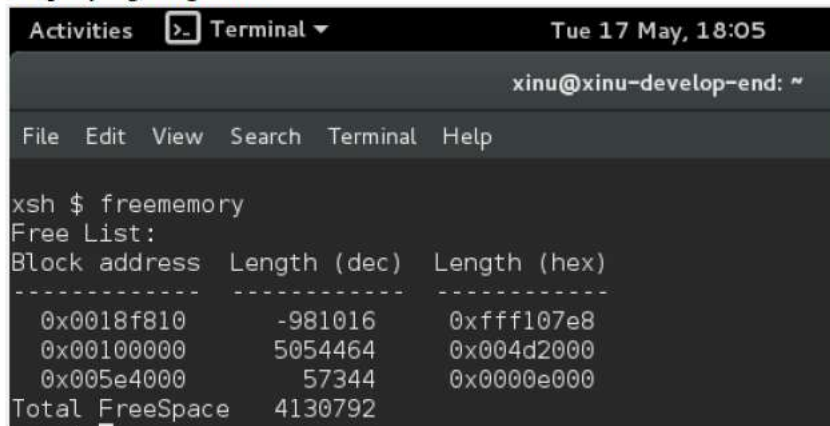
JURNAL/TP

Soal:

Jurnal

1. [80 poin] Buatlah perintah baru bernama **freememory** yang memiliki dua fungsi berikut:
 - a. [40 poin] Menampilkan seluruh free memory block yang tercatat dalam free memory list pada Xinu.
 - b. [40 poin] Menghitung dan menampilkan total ukuran free memory berdasarkan seluruh block yang ada pada list tersebut.

Output yang diinginkan:



```
Activities > Terminal Tue 17 May, 18:05
xinu@xinu-develop-end: ~
File Edit View Search Terminal Help

xsh $ freememory
Free List:
Block address Length (dec) Length (hex)
-----
0x0018f810 -981016 0xffff107e8
0x00100000 5054464 0x004d2000
0x005e4000 57344 0x0000e000
Total FreeSpace 4130792
```

2. [4 poin per subsoal] Jawablah pertanyaan berikut:
 - a. Mengapa Xinu memisahkan data segment dan BSS segment?
 - b. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
 - c. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
 - d. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
 - e. Apa tantangan utama dalam penggunaan heap di Xinu?

Jawaban:

1. Program freememory melakukan traversal pada linked list memlist, menampilkan setiap free memory block beserta ukurannya, kemudian menjumlahkan seluruh ukuran blok untuk memperoleh total free memory yang tersedia.

Kalau setelah mengganti %d menjadi %u masih muncul nilai negatif, kirim isi lengkap xsh_freememory.c, karena kemungkinan struktur memblk pada versi Xinu kamu menggunakan field yang sedikit berbeda.

A.



```
/*
 * Nama : Farhan Kurniawan
 * NIM : ISI NIM
 */

#include <xinu.h>
#include <stdio.h>

shellcmd xsh_freememory(int nargs, char *args[])
{
    struct memblk *block;
    uint32 total = 0;

    printf("Free Memory Blocks:\n");
    printf("Block Address    Size(Bytes)\n");
    printf("-----\n");

    for (block = memlist.mnext; block != NULL; block = block->mnext)
    {
        printf("0x%08x    %10d\n",
            block,
            block->mlength);

        total += block->mlength;
    }

    printf("-----\n");
    printf("Total Free Memory = %d bytes\n", total);
}
```

Activities gedit Tue 16 Jun, 22:53

shprototypes.h
~/xinu/include

```
/* in file xsh_ls.c */
extern shellcmd xsh_ls      (int32, char *[]);

/* in file xsh_led.c */
extern shellcmd xsh_led     (int32, char *[]);

/* in file xsh_memdump.c */
extern shellcmd xsh_memdump (int32, char *[]);

/* in file xsh_memstat.c */
extern shellcmd xsh_memstat (int32, char *[]);
extern shellcmd xsh_freememory(int32, char *[]);

/* in file xsh_netinfo.c */
extern shellcmd xsh_netinfo (int32, char *[]);

/* in file xsh_ns.c */
extern shellcmd xsh_ns      (int32, char *[]);

/* in file xsh_nvram.c */
extern shellcmd xsh_nvram   (int32, char *[]);

/* in file xsh_ping.c */
extern shellcmd xsh_ping    (int32, char *[]);

/* in file xsh_ps.c */
```

C/C++/ObjC Header Tab Width: 8 Ln 50, Col 1 INS

Activities gedit Tue 16 Jun, 22:54

shell.c
~/xinu/shell

```
/* Table of Xinu shell commands and the function associated with each */
/*****
const struct cmdent cmdtab[] = {
    {"argecho", xsh_argecho},
    {"arp", xsh_arp},
    {"cat", xsh_cat},
    {"clear", xsh_clear},
    {"date", xsh_date},
    {"devdump", xsh_devdump},
    {"echo", xsh_echo},
    {"help", xsh_help},
    {"ls", xsh_ls},
    {"kill", xsh_kill},
    {"memdump", xsh_memdump},
    {"freememory", xsh_freememory},
    {"memstat", xsh_memstat},
    {"ns", xsh_ns},
    {"netinfo", xsh_netinfo},
    {"ping", xsh_ping},
    {"ps", xsh_ps},
    {"sleep", xsh_sleep},
    {"tee", xsh_tee},
    {"udp", xsh_udpdump},
    {"udpecho", xsh_udpecho},
    {"udpserver", xsh_udpserver},
    {"uptime", xsh_uptime},
    {"?", xsh_help}
}
```

C Tab Width: 8 Ln 1, Col 1 INS

B.

```
Activities Terminal Tue 16 Jun, 22:44
xinu@xinu-develop-end: ~/xinu/compile

-----2311104073-----

Welcome to Xinu!

xsh $ hel
command hel not found
xsh $ help

shell commands are:

argecho    date      ls         memstat    ps         udpecho
arp         devdump  kill       ns          sleep      udpserver
cat         echo     memdump   netinfo    tee        uptime
clear      help     freememory ping        udp        ?

xsh $ freememory
Free Memory Blocks:
Block Address  Size(Bytes)
-----
0x0018e118    -975128
0x00100000     5070848
0x005e8000      57344
-----
Total Free Memory = 4153064 bytes
xsh $

CTRL-A Z for help | 115200 BN1 | NOR | Minicom 2.7 | VT102 | Online 0:3 | ttyS0
```

2.

- A. Mengapa Xinu memisahkan data segment dan BSS segment?
Data segment menyimpan variabel global yang sudah memiliki nilai awal, sedangkan BSS segment menyimpan variabel global yang belum diinisialisasi. Pemisahan ini membuat penggunaan memori lebih efisien karena variabel pada BSS tidak perlu disimpan nilainya di file program.
- B. Bagaimana alokasi dan dealokasi memori selama eksekusi memengaruhi ukuran free space?
Saat memori dialokasikan, ukuran free space berkurang karena sebagian memori digunakan proses. Sebaliknya, ketika memori didealokasi atau dibebaskan, ukuran free space bertambah kembali sehingga dapat digunakan oleh proses lain.
- C. Mengapa penggunaan heap lebih berpotensi menimbulkan masalah dibandingkan stack?
Heap lebih rentan mengalami fragmentasi memori akibat alokasi dan dealokasi yang tidak teratur. Selain itu, programmer harus mengelola heap secara manual sehingga lebih berisiko terjadi memory leak atau kesalahan pengelolaan memori.
- D. Mengapa Xinu menggunakan struktur linked list untuk menyimpan free block?
Linked list memudahkan penambahan, penghapusan, dan penggabungan blok memori bebas secara dinamis. Struktur ini juga lebih fleksibel karena ukuran dan jumlah blok memori bebas dapat berubah selama sistem berjalan.

E. Apa tantangan utama dalam penggunaan heap di Xinu?

Tantangan utamanya adalah fragmentasi memori, yaitu kondisi ketika ruang kosong tersedia tetapi terpecah menjadi blok-blok kecil sehingga sulit digunakan untuk alokasi memori yang lebih besar. Selain itu, pengelolaan heap yang tidak tepat dapat menyebabkan memory leak.